

Sprawozdanie z Listy 2 (Laboratorium)

Obliczenia Naukowe

Agnieszka Głuszkiewicz

1 Zadanie 1: Wpływ niewielkich zmian danych na obliczenia

1.1 Opis problemu

Celem zadania było zbadanie wpływu niewielkich zmian danych na dokładność obliczeń. Wykonałam powtórzenie zadania 5 z listy 1 (obliczanie iloczynu skalarnego $\mathbf{x} \cdot \mathbf{y}$ na 4 sposoby różniące się kolejnością sumowania, w precyzjach Float32 i Float64), wprowadzając minimalne zmiany do wektora $\mathbf{x}$ - w tym przypadku poprzez usunięcie ostatniej cyfry z x_4 i x_5 .

1.2 Rozwiązanie

Wektory użyte w eksperymencie to:

- **Wektor $\mathbf{x}$ (niezmieniony):**

$$\mathbf{x} = [2.718281828, -3.141592654, 1.414213562, 0.5772156649, 0.3010299957]$$

- **Wektor zmodyfikowany $\mathbf{x}'$:**

$$\mathbf{x}' = [2.718281828, -3.141592654, 1.414213562, 0.577215664, 0.301029995]$$

- **Wektor $\mathbf{y}$ (niezmieniony):**

$$\mathbf{y} = [1486.2497, 878366.9879, -22.37492, 4773714.647, 0.000185049]$$

Obliczyłam iloczyn skalarny wektorów $\mathbf{x}'$ i $\mathbf{y}$ przy użyciu czterech metod: sumowania w przód, w tył oraz sumowania po uprzednim posortowaniu składników od największego do najmniejszego oraz od najmniejszego do największego. Porównałam wyniki z wynikami obliczeń dla iloczynu skalarnego $\mathbf{x}$ i $\mathbf{y}$.

1.3 Wyniki i interpretacja

Właściwa wartość sumy: $1.00657107000000 \times 10^{-11}$.

Table 1: Wyniku obliczania iloczynu skalarnego: porównanie oryginalnych i zmodyfikowanych danych

Typ	Algorytm	Wynik $\mathbf{x} \cdot \mathbf{y}$ (Oryginalny)	Wynik $\mathbf{x}' \cdot \mathbf{y}$ (Zmodyfikowany)
Float32	W przód (Forward)	-0.4999443	-0.4999443
	W tył (Backward)	-0.4543457	-0.4543457
	Max $\rightarrow$ Min	-0.5	-0.5
	Min $\rightarrow$ Max	-0.5	-0.5
Float64	W przód (Forward)	$1.0251881368296672 \times 10^{-10}$	$-4.296342739891585 \times 10^{-3}$
	W tył (Backward)	$-1.5643308870494366 \times 10^{-10}$	$-4.296342998713953 \times 10^{-3}$
	Max $\rightarrow$ Min	0.0	$-4.296342842280865 \times 10^{-3}$
	Min $\rightarrow$ Max	0.0	$-4.296342842280865 \times 10^{-3}$

Table 2: Błędy względne iloczynu skalarnego: porównanie oryginalnych i zmodyfikowanych danych

Typ	Algorytm Sumowania	Błąd Względny ($x \cdot y$)	Błąd Względny ($x' \cdot y$)
Float32	W przód (Forward)	$4.9668057661282845 \times 10^{10}$	$4.9668057661282845 \times 10^{10}$
	W tył (Backward)	$4.51379655800096 \times 10^{10}$	$4.51379655800096 \times 10^{10}$
	Max $\rightarrow$ Min	$4.967359135306107 \times 10^{10}$	$4.967359135306107 \times 10^{10}$
	Min $\rightarrow$ Max	$4.967359135306107 \times 10^{10}$	$4.967359135306107 \times 10^{10}$
Float64	W przód (Forward)	11.184955313981627	$4.2682954615672344 \times 10^8$
	W tył (Backward)	14.541186645165915	$4.2682957186999655 \times 10^8$
	Max $\rightarrow$ Min	1.0	$4.268295563288099 \times 10^8$
	Min $\rightarrow$ Max	1.0	$4.268295563288099 \times 10^8$

Dla Float32 zmiana była zbyt mała w stosunku do precyzji, co spowodowało **brak zmian** w wynikach i błędach.

Dla Float64, gdzie precyzja jest dokładniejsza, wpływ małej zmiany danych był znaczący.

- Wyniki numeryczne **zmieniły rząd wielkości** (niektóre z $\approx 10^{-10}$ na $\approx 10^{-3}$).
- Błąd względny **zdecydowanie wzrósł** (z wartości 1.0 do $\approx 10^8$), co oznacza znaczną utratę dokładności.
- Korzyści algorytmów sortujących zostały zniwelowane przez błąd wynikający ze zmiany danych wejściowych.

1.4 Wnioski

Eksperyment dowodzi, że problem obliczania iloczynu skalarnego w tym przypadku jest **źle uwarunkowany**. Niewielka zmiana danych wejściowych wywołała bardzo duży błąd w wyniku końcowym, prawie całkowicie maskując różnice w wynikach przy różnych algorytmach sumowania.

2 Zadanie 2: Analiza funkcji $f(x) = e^x \ln(1 + e^{-x})$

2.1 Opis problemu

Zadanie polegało na wizualizacji funkcji $f(x) = e^x \ln(1 + e^{-x})$, obliczeniu granicy $\lim_{x \rightarrow \infty} f(x)$ oraz analizie zjawiska obserwowanego na wykresach.

2.2 Rozwiązanie

Granica funkcji, wyliczona przy pomocy odpowiednich przekształceń i zastosowania reguły de l'Hôpitala:

$$\begin{aligned}
 \lim_{x \rightarrow \infty} f(x) &= \lim_{x \rightarrow \infty} e^x \ln(1 + e^{-x}) = \lim_{x \rightarrow \infty} \frac{\ln(1 + e^{-x})}{e^{-x}} = (l'Hopital) = \lim_{x \rightarrow \infty} \frac{(\ln(1 + e^{-x}))'}{(e^{-x})'} = \\
 &= \lim_{x \rightarrow \infty} \frac{\frac{1}{1+e^{-x}} \cdot (-e^{-x})}{-e^{-x}} = \lim_{x \rightarrow \infty} \frac{1}{1 + e^{-x}} = \frac{1}{1 + 0} = 1
 \end{aligned}$$

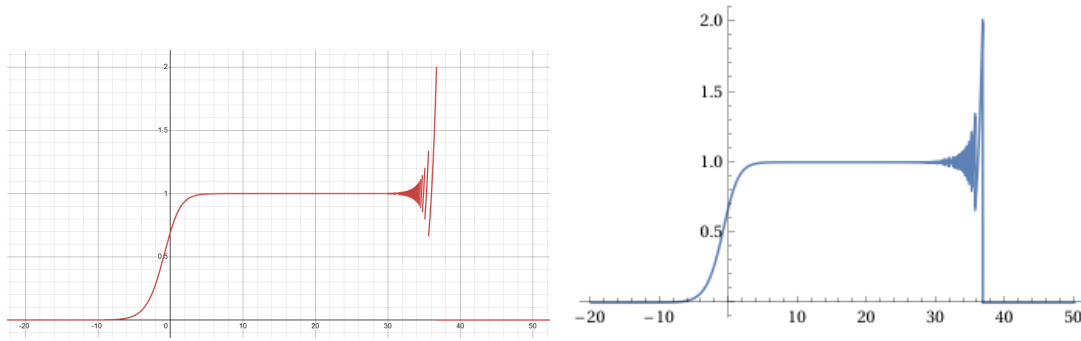


Figure 1: Wizualizacja funkcji $f(x) = e^x \ln(1 + e^{-x})$ (przykłady z Desmos i Wolfram Alpha).

2.3 Wyniki i interpretacja

Wygenerowane wykresy pokazują, że funkcja dąży do wartości 1 dla $x \in [0, 30]$. Dla $x > 30$ na obu wykresach pojawiają się nieregularne oscylacje, a po chwili wartość spada do 0 i taka już pozostaje.

Zaobserwowane zjawisko jest konsekwencją błędu zaokrąglenia, który wynika z mnożenia liczb mocno różniących się wielkością. Dla dużych wartości x , działanie mnożenia bardzo dużej liczby (e^x) przez bardzo mały czynnik ($\ln(1 + e^{-x})$) jest obciążone znaczącym błędem, co objawia się chaotycznymi oscylacjami. Z czasem, gdy precyzja określona dla programu graficznego nie pozwala już zapisać liczby tak małej jak e^{-x} , przypisuje się jej wartość zero i $e^x \ln(1 + e^{-x})$ zmienia się w $e^x \ln(1) = e^x \cdot 0 = 0$, mimo że granica analityczna wynosi 1.

2.4 Wnioski

Wyniki na wykresach, odbiegające od analitycznie wyliczonej granicy $\lim_{x \rightarrow \infty} f(x) = 1$, wskazują na **niestabilność numeryczną** zastosowanego wzoru. Obserwowane oscylacje i błędy są spowodowane utratą cyfr znaczących w czynniku z logarytmem, co jest następnie wzmacniane przez mnożenie przez rosnące e^x .

3 Zadanie 3: Analiza stabilności $Ax = b$

3.1 Opis problemu

Celem było zbadanie stabilności numerycznej dwóch metod rozwiązywania układów $Ax = b$: **eliminacji Gaussa** ($A \setminus b$) oraz **odwracania macierzy** ($A^{-1}b$). Eksperyment przeprowadziłam na macierzach Hilberta (H_n) i losowych macierzach uwarunkowanych (R_n) o rosnącym wskaźniku uwarunkowania $c \in [1, 10^{16}]$, przy czym obliczałam błędy względne. Znane rozwiązanie dokładne to $\mathbf{x} = (1, 1, \dots, 1)^T$.

3.2 Rozwiązanie

Macierze generowałam za pomocą funkcji `hilb(n)` i `matcond(n, c)` dołączonych do zadania. Porównałam błędy względne uzyskane z metody $A \setminus b$ z metodą $A^{-1}b$. Sprawdzałam też wartości wskaźnika uwarunkowania wygenerowanej macierzy i jej rząd.

3.3 Wyniki i interpretacja

3.3.1 Macierze Hilberta (H_n)

W macierzach Hilberta wskaźnik uwarunkowania $\text{cond}(H_n)$ i błąd względny rosną bardzo szybko. Eliminacja Gaussa ($A \setminus b$) jest bardziej stabilna, ale obie metody wykazują pogarszającą się dokładność wraz ze wzrostem n .

Table 3: Błędy względne dla Macierzy Hilberta $\mathbf{H}_n$

n	$\text{rank}(\mathbf{H}_n)$	$\text{cond}(\mathbf{H}_n)$	Błąd Wzgl. (Gauss: $\mathbf{A} \setminus \mathbf{b}$)	Błąd Wzgl. (Inv: $\mathbf{A}^{-1}\mathbf{b}$)
2	2	1.9281×10^1	$5.661048867003676 \times 10^{-16}$	$1.4043333874306803 \times 10^{-15}$
3	3	5.2406×10^2	$8.022593772267726 \times 10^{-15}$	0.0
4	4	1.5514×10^4	$4.137409622430382 \times 10^{-14}$	0.0
5	5	4.7661×10^5	$1.6828426299227195 \times 10^{-12}$	$3.3544360584359632 \times 10^{-12}$
6	6	1.4951×10^7	$2.618913302311624 \times 10^{-10}$	$2.0163759404347654 \times 10^{-10}$
7	7	4.7537×10^8	$1.2606867224171548 \times 10^{-8}$	$4.713280397232037 \times 10^{-9}$
8	8	1.5258×10^{10}	$6.124089555723088 \times 10^{-8}$	$3.07748390309622 \times 10^{-7}$
9	9	4.9315×10^{11}	$3.8751634185032475 \times 10^{-6}$	$4.541268303176643 \times 10^{-6}$
10	10	1.6024×10^{13}	$8.67039023709691 \times 10^{-5}$	$2.501493411824886 \times 10^{-4}$

3.3.2 Losowe macierze uwarunkowane ($\mathbf{R}_n$)

Zestawienie według celowanego uwarunkowania c pokazuje, że błąd względny rośnie wprost proporcjonalnie do c , niezależnie od rozmiaru macierzy n .

Table 4: Błędy względne dla macierzy $\mathbf{R}_n$

c	n	$\text{rank}(\mathbf{R}_n)$	$\text{cond}(\mathbf{R}_n)$	Błąd Wzgl. (Gauss: $\mathbf{A} \setminus \mathbf{b}$)	Błąd Wzgl. (Inv: $\mathbf{A}^{-1}\mathbf{b}$)
1.0×10^0	5	5	$1.0000000000000004 \times 10^0$	$9.930136612989092 \times 10^{-17}$	$7.021666937153402 \times 10^{-17}$
	10	10	$1.0000000000000007 \times 10^0$	$3.2368285245694683 \times 10^{-16}$	$2.3811631099687444 \times 10^{-16}$
	20	20	$1.0000000000000016 \times 10^0$	$7.069776086746845 \times 10^{-16}$	$5.324442579404919 \times 10^{-16}$
1.0×10^1	5	5	$1.0000000000000005 \times 10^1$	$1.719950113979703 \times 10^{-16}$	$2.808666774861361 \times 10^{-16}$
	10	10	$1.0000000000000005 \times 10^1$	$3.813741331030777 \times 10^{-16}$	$3.940900794429957 \times 10^{-16}$
	20	20	$1.0000000000000000 \times 10^1$	$8.506076425164523 \times 10^{-16}$	$4.447825579847492 \times 10^{-16}$
1.0×10^3	5	5	$1.0000000000000244 \times 10^3$	$3.168861251852853 \times 10^{-14}$	$4.258072221745391 \times 10^{-14}$
	10	10	$1.0000000000001424 \times 10^3$	$8.94401622147837 \times 10^{-16}$	$1.006428041294448 \times 10^{-14}$
	20	20	$1.000000000000167 \times 10^3$	$2.933174772761872 \times 10^{-15}$	$5.256410677008212 \times 10^{-15}$
1.0×10^7	5	5	$1.0000000001720143 \times 10^7$	$1.3762347368514775 \times 10^{-10}$	$1.8544416318268037 \times 10^{-10}$
	10	10	$9.99999999529068 \times 10^6$	$9.883468330357592 \times 10^{-11}$	$1.2234140588468305 \times 10^{-10}$
	20	20	$9.999999992713638 \times 10^6$	$5.626873939195996 \times 10^{-10}$	$5.187839415943533 \times 10^{-10}$
1.0×10^{12}	5	5	$1.0000012297517992 \times 10^{12}$	$1.2444752303462837 \times 10^{-5}$	$1.299239072107422 \times 10^{-5}$
	10	10	$1.0000424034172561 \times 10^{12}$	$2.5199278094560093 \times 10^{-5}$	$1.6382670310214727 \times 10^{-5}$
	20	20	$1.0000531800744414 \times 10^{12}$	$8.775480952606155 \times 10^{-7}$	$5.8942936479043745 \times 10^{-6}$
1.0×10^{16}	5	4	$4.157691511497868 \times 10^{16}$	$5.315824801057405 \times 10^{-1}$	$4.671326263760646 \times 10^{-1}$
	10	9	$2.071170975025979 \times 10^{16}$	$4.026525450428533 \times 10^{-1}$	$4.3284352325638414 \times 10^{-1}$
	20	19	$1.811230274003154 \times 10^{16}$	$3.097592274521324 \times 10^{-1}$	$3.087358751039015 \times 10^{-1}$

3.4 Wnioski

1. Dla rozwiązywania układu równań liniowych z macierzą Hilberta wraz ze wzrostem rozmiaru macierzy bardzo szybko rośnie zarówno wskaźnik uwarunkowania, jak i błąd, więc już dla niewielkich macierzy Hilberta zadanie jest źle uwarunkowane.
2. Rozwiązywanie układu równań liniowych za pomocą eliminacji Gaussa ($\mathbf{A} \setminus \mathbf{b}$), jest stabilniejsze niż użycie odwracania macierzy ($\mathbf{A}^{-1}\mathbf{b}$).
3. Gdy macierz $\mathbf{A}$ ma wysoki wskaźnik uwarunkowania $\text{cond}(\mathbf{A})$, zadanie rozwiązania układu równań liniowych $Ax = b$ jest **źle uwarunkowane** i nie da się go rozwiązać z wysoką precyzją w arytmetyce Float64.

4 Zadanie 4: "Złośliwy wielomian" Wilkinsona

4.1 Opis problemu

Zadanie polegało na analizie, jak błąd w reprezentacji współczynników w arytmetyce Float64 wpływa na dokładność obliczanych pierwiastków wielomianu Wilkinsona $P(x)$ (postać ogólna) oraz $p(x) = \prod_{k=1}^{20} (x - k)$ (postać iloczynowa). Należało też przeprowadzić eksperyment, zmieniając współczynnik c_{19} o wartość $\delta = -2^{-23}$.

4.2 Rozwiązanie

Policzyłam pierwiastki, wykorzystując funkcję `roots` z pakietu `Polynomials` w Julii. Następnie sprawdziłam obliczone pierwiastki, licząc $|P(z_k)|$, $|p(z_k)|$ oraz $|z_k - k|$. Powtórzyłam ten proces dla zmienionego współczynnika c_{19} . Obliczenia przeprowadziłam w Float64.

4.3 Wyniki i interpretacja

Oryginalny wielomian:

Wielomian $P(x)$ jest już niestabilny z powodu błędów zaokrągleń maszynowych. Wartości $|P(z_k)|$, $|p(z_k)|$ rosną wraz ze wzrostem k , coraz bardziej odbiegając od wartości 0 (z definicji przypisywanej pierwiastkowi wielomianu).

Zmieniony wielomian:

Minimalna zmiana współczynnika c_{19} prowadzi do sporego wzrostu błędu i pojawienia się pierwiastków zespolonych.

Table 5: Wyniki dla oryginalnego wielomianu

$\mathbf{k}$	$\mathbf{z}_k$	$ \mathbf{P}(\mathbf{z}_k) $	$ \mathbf{p}(\mathbf{z}_k) $	$ \mathbf{z}_k - \mathbf{k} $
1	0.999999999998084	$2.3323616390897252 \times 10^4$	$2.3310180819556477 \times 10^4$	$1.9162449405030202 \times 10^{-13}$
2	2.0000000000114264	$6.4613550791712885 \times 10^4$	$7.315618130995684 \times 10^4$	$1.1426415369442111 \times 10^{-11}$
3	2.999999998168487	$1.8851098984644806 \times 10^4$	$1.3028920977428104 \times 10^5$	$1.8315127192636282 \times 10^{-10}$
4	3.99999983818672	$2.6359390809003003 \times 10^6$	$2.0313511850413054 \times 10^6$	$1.6181327833209025 \times 10^{-8}$
5	5.000000688670983	$2.3709842874839526 \times 10^7$	$2.1613359049100634 \times 10^7$	$6.88670983350903 \times 10^{-7}$
6	5.999988371602095	$1.2641076289358065 \times 10^8$	$1.2165063267640364 \times 10^8$	$1.162839790502801 \times 10^{-5}$
7	7.000112910766231	$5.2301629899144447 \times 10^8$	$5.061885949319773 \times 10^8$	$1.1291076623098917 \times 10^{-4}$
8	7.999279406281878	$1.798432141726085 \times 10^9$	$1.7402728325721796 \times 10^9$	$7.205937181220534 \times 10^{-4}$
9	9.003273831140774	$5.121881552672067 \times 10^9$	$5.263758084354485 \times 10^9$	$3.273831140774064 \times 10^{-3}$
10	9.989265687778465	$1.4157542666785017 \times 10^{10}$	$1.4147808356077827 \times 10^{10}$	$1.0734312221535092 \times 10^{-2}$
11	11.027997558569794	$3.586354765112257 \times 10^{10}$	$3.692632803664625 \times 10^{10}$	$2.7997558569794023 \times 10^{-2}$
12	11.94827395840048	$8.510931555828575 \times 10^{10}$	$8.162184753413098 \times 10^{10}$	$5.1726041599520656 \times 10^{-2}$
13	13.082031971969954	$2.2136146301419052 \times 10^{11}$	$2.04374354285492 \times 10^{11}$	$8.203197196995404 \times 10^{-2}$
14	13.906800565193148	$3.812024574451268 \times 10^{11}$	$3.8519295444834576 \times 10^{11}$	$9.319943480685211 \times 10^{-2}$
15	15.081439299377482	$8.809029239560208 \times 10^{11}$	$9.126025022282091 \times 10^{11}$	$8.14392993774824 \times 10^{-2}$
16	15.942404318674466	$1.6747434633806333 \times 10^{12}$	$1.6751148968378867 \times 10^{12}$	$5.759568132553383 \times 10^{-2}$
17	17.026861831476396	$3.3067827086376123 \times 10^{12}$	$3.5115331717998135 \times 10^{12}$	$2.6861831476395537 \times 10^{-2}$
18	17.99048462339055	$6.166202940769282 \times 10^{12}$	$6.644365795060319 \times 10^{12}$	$9.515376609449788 \times 10^{-3}$
19	19.001981084996206	$1.406783619602919 \times 10^{13}$	$1.274643243051272 \times 10^{13}$	$1.981084996206306 \times 10^{-3}$
20	19.999803908064397	$3.284992217648231 \times 10^{13}$	$2.3837033672895914 \times 10^{13}$	$1.9609193560299332 \times 10^{-4}$

Table 6: Wyniki dla wielomianu ze zmienionym współczynnikiem

$\mathbf{k}$	$\mathbf{z}_k$	$ \mathbf{P}(\mathbf{z}_k) $	$ \mathbf{p}(\mathbf{z}_k) $	$ \mathbf{z}_k - \mathbf{k} $
1	0.999999999999805	2.16893617×10^3	2.37693617×10^3	$1.95399252 \times 10^{-14}$
2	1.999999999985736	2.99484390×10^4	9.13243896×10^3	$1.42641454 \times 10^{-12}$
3	3.000000000105087	2.39010535×10^5	7.47562852×10^4	$1.05087050 \times 10^{-10}$
4	3.9999999950066143	9.39293805×10^5	6.26853364×10^5	$4.99338570 \times 10^{-9}$
5	5.000000034712704	7.44868040×10^6	1.08942987×10^6	$3.47127038 \times 10^{-8}$
6	6.000005852511414	1.46893325×10^7	6.12250864×10^7	$5.85251141 \times 10^{-6}$
7	6.999704466216799	5.81794640×10^7	1.32529817×10^9	$2.95533783 \times 10^{-4}$
8	8.007226654064777	1.39542059×10^8	$1.73807347 \times 10^{10}$	$7.22665406 \times 10^{-3}$
9	8.917396943382494	2.45961776×10^8	$1.34872915 \times 10^{11}$	$8.26030566 \times 10^{-2}$
10	$10.09529034477879 + 0.64327709i$	2.29101856×10^9	$1.48243475 \times 10^{12}$	$6.50296597 \times 10^{-1}$
11	$10.09529034477879 - 0.64327709i$	2.29101856×10^9	$1.48243475 \times 10^{12}$	1.11009233×10^0
12	$11.793588728372308 + 1.65225355i$	$2.07769079 \times 10^{10}$	$3.29395824 \times 10^{13}$	1.66509681×10^0
13	$11.793588728372308 - 1.65225355i$	$2.07769079 \times 10^{10}$	$3.29395824 \times 10^{13}$	2.04581767×10^0
14	$13.99233053734825 + 2.51881964i$	$9.39073060 \times 10^{10}$	$9.54541282 \times 10^{14}$	2.51883132×10^0
15	$13.99233053734825 - 2.51881964i$	$9.39073060 \times 10^{10}$	$9.54541282 \times 10^{14}$	2.71290437×10^0
16	$16.73073008036981 + 2.81262730i$	$9.59235656 \times 10^{11}$	$2.74207058 \times 10^{16}$	2.82548732×10^0
17	$16.73073008036981 - 2.81262730i$	$9.59235656 \times 10^{11}$	$2.74207058 \times 10^{16}$	3.71960207×10^0
18	$19.50243895868367 + 1.94033202i$	$5.05046740 \times 10^{12}$	$4.25245471 \times 10^{17}$	2.45401939×10^0
19	$19.50243895868367 - 1.94033202i$	$5.05046740 \times 10^{12}$	$4.25245471 \times 10^{17}$	2.00432863×10^0
20	20.84690887410499	$4.85865313 \times 10^{12}$	$1.37436721 \times 10^{18}$	$8.46908874 \times 10^{-1}$

4.4 Wnioski

1. Zadanie wyznaczania pierwiastków wielomianu Wilkinsona jest **bardzo źle uwarunkowane**. Nawet minimalna zmiana współczynnika prowadzi do ogromnego błędu w samych pierwiastkach (w tym pojawienia się liczb zespolonych) oraz błędu bezwzględnego rzędu 10^0 .
2. Reprezentowanie wielomianów w postaci ogólnej i iloczynowej daje inne wyniki, co wskazuje na istotną rolę użytej reprezentacji.

5 Zadanie 5: Stabilność numeryczna modelu logistycznego

5.1 Opis problemu

Celem było zbadanie zachowania równania rekurencyjnego modelu logistycznego $p_{n+1} = p_n + rp_n(1 - p_n)$ dla $p_0 = 0.01$ i $r = 3$. Należało przeprowadzić dwa eksperymenty: (1) analiza wpływu celowego obciążenia ($p_{10} \rightarrow 0.722$ w Float32) oraz (2) porównanie precyzji (Float32 vs. Float64).

5.2 Rozwiązanie

Zaimplementowałam równanie rekurencyjne w Julii. W eksperymencie 1 zastosowałam celowe obciążenie p_{10} dla arytmetyki Float32, a w eksperymencie 2 porównałam wyniki w arytmetykach Float32 i Float64 przez 40 iteracji.

5.3 Wyniki i interpretacja

5.3.1 Eksperyment 1: Obciążenie wyniku w trakcie (Float32)

Poniższa tabela ilustruje rozbieżność wyników po wprowadzeniu celowego obciążenia w wyniku obciążenia p_{10} do 0.722. Kolejne kumulujące się różnice w błędach zaokrągleń wynikające z tego obciążenia są wystarczająco duże, aby doprowadzić do dużej rozbieżności między obliczonymi wartościami, pomimo używania tej samej precyzji.

Table 7: Wartości wywołań modelu logistycznego w Float32 (baza vs. celowe obciążenie)

n	$\mathbf{p_n}$ (F32 Baza)	$\mathbf{p_n}$ (F32 Celowe obciążenie)	Różnica Base – Obciążenie
0	0.01	0.01	0.0
1	0.0397	0.0397	0.0
2	0.15407173	0.15407173	0.0
3	0.5450726	0.5450726	0.0
4	1.2889781	1.2889781	0.0
5	0.1715188	0.1715188	0.0
6	0.5978191	0.5978191	0.0
7	1.3191134	1.3191134	0.0
8	0.056273222	0.056273222	0.0
9	0.21559286	0.21559286	0.0
10	0.7229306	0.722	9.306073×10^{-4}
11	1.3238364	1.3241479	3.1149387×10^{-4}
12	0.037716985	0.036488414	1.2285709×10^{-3}
13	0.14660022	0.14195944	4.6407730×10^{-3}
14	0.521926	0.50738037	$1.45456195 \times 10^{-2}$
15	1.2704837	1.2572169	1.3266802×10^{-2}
16	0.2395482	0.28708452	4.7536314×10^{-2}
17	0.7860428	0.9010855	1.1504269×10^{-1}
18	1.2905813	1.1684768	$1.22104526 \times 10^{-1}$
19	0.16552472	0.577893	4.1236830×10^{-1}
20	0.5799036	1.3096911	7.2978747×10^{-1}
21	1.3107498	0.09289217	1.2178576×10^0
22	0.088804245	0.34568182	2.5687757×10^{-1}
23	0.3315584	1.0242395	6.9268113×10^{-1}
24	0.9964407	0.94975823	4.6682477×10^{-2}
25	1.0070806	1.0929108	8.5830210×10^{-2}
26	0.9856885	0.7882812	1.9740730×10^{-1}
27	1.0280086	1.2889631	2.6095450×10^{-1}
28	0.9416294	0.17157483	7.7005457×10^{-1}
29	1.1065198	0.59798557	5.0853425×10^{-1}
30	0.7529209	1.3191822	5.6626123×10^{-1}
31	1.3110139	0.05600393	1.2550100×10^0
32	0.0877831	0.21460639	1.2682329×10^{-1}
33	0.3280148	0.7202578	3.9224303×10^{-1}
34	0.9892781	1.3247173	3.3543920×10^{-1}
35	1.021099	0.034241438	9.8685753×10^{-1}
36	0.95646656	0.13344833	8.2301820×10^{-1}
37	1.0813814	0.48036796	6.0101350×10^{-1}
38	0.81736827	1.2292118	4.1184354×10^{-1}
39	1.2652004	0.3839622	8.8123816×10^{-1}
40	0.25860548	1.093568	8.3496250×10^{-1}

5.3.2 Eksperyment 2: Porównanie precyzji (F32 vs F64)

Poniższa tabela ilustruje rozbieżność wyników wynikającą wyłącznie z różnicy precyzji arytmetycznej. Różnice w błędach zaokrągleń (które są większe w F32) są wystarczająco duże, aby doprowadzić do sporej rozbieżności między wartościami obliczonymi w obu precyzjach.

Table 8: Wartości wywołań modelu logistycznego w Float32 i Float64

n	$\mathbf{P}_n$ (F32)	$\mathbf{P}_n$ (F64)	Różnica $ \mathbf{F64} - \mathbf{F32} $
0	1.0000000×10^{-2}	$1.0000000000000000 \times 10^{-2}$	$2.23517418 \times 10^{-10}$
1	3.9700000×10^{-2}	$3.9700000000000000 \times 10^{-2}$	$1.47819519 \times 10^{-9}$
2	1.5407173×10^{-1}	$1.54071730000000002 \times 10^{-1}$	$3.35552214 \times 10^{-9}$
3	5.4507260×10^{-1}	$5.450726260444213 \times 10^{-1}$	$1.08977843 \times 10^{-8}$
4	1.2889781×10^0	$1.2889780011888006 \times 10^0$	$9.86341975 \times 10^{-8}$
5	1.7151880×10^{-1}	$1.7151914210917552 \times 10^{-1}$	$3.39466353 \times 10^{-7}$
6	5.9781910×10^{-1}	$5.978201201070994 \times 10^{-1}$	$1.03021757 \times 10^{-6}$
7	1.3191134×10^0	$1.3191137924137974 \times 10^0$	$4.18657389 \times 10^{-7}$
8	5.6273222×10^{-2}	$5.6271577646256565 \times 10^{-2}$	$1.64432335 \times 10^{-6}$
9	2.1559286×10^{-1}	$2.1558683923263022 \times 10^{-1}$	$6.02194291 \times 10^{-6}$
10	7.2293060×10^{-1}	$7.229143011795730 \times 10^{-1}$	$1.63090003 \times 10^{-5}$
11	1.3238364×10^0	$1.3238419441684408 \times 10^0$	$5.49836003 \times 10^{-6}$
12	3.7716985×10^{-2}	$3.769529725473175 \times 10^{-2}$	$2.16874941 \times 10^{-5}$
13	1.4660022×10^{-1}	$1.4651838271355924 \times 10^{-1}$	$8.18339136 \times 10^{-5}$
14	5.2192600×10^{-1}	$5.216706214352460 \times 10^{-1}$	$2.55377895 \times 10^{-4}$
15	1.2704837×10^0	$1.2702617739350768 \times 10^0$	$2.21958288 \times 10^{-4}$
16	2.3954820×10^{-1}	$2.4035217277824272 \times 10^{-1}$	$8.03972778 \times 10^{-4}$
17	7.8604280×10^{-1}	$7.881011902353041 \times 10^{-1}$	$2.05839024 \times 10^{-3}$
18	1.2905813×10^0	$1.2890943027903075 \times 10^0$	$1.48704277 \times 10^{-3}$
19	1.6552472×10^{-1}	$1.7108484670194324 \times 10^{-1}$	$5.56012556 \times 10^{-3}$
20	5.7990360×10^{-1}	$5.965293124946907 \times 10^{-1}$	$1.66257125 \times 10^{-2}$
21	1.3107498×10^0	$1.3185755879825978 \times 10^0$	$7.82578800 \times 10^{-3}$
22	8.8804245×10^{-2}	$5.8377608259430724 \times 10^{-2}$	$3.04266367 \times 10^{-2}$
23	3.3155840×10^{-1}	$2.2328659759944824 \times 10^{-1}$	$1.08271809 \times 10^{-1}$
24	9.9644070×10^{-1}	$7.435756763951792 \times 10^{-1}$	$2.52865024 \times 10^{-1}$
25	1.0070806×10^0	$1.315588346001072 \times 10^0$	$3.08507791 \times 10^{-1}$
26	9.8568850×10^{-1}	$7.003529560277899 \times 10^{-2}$	$9.15653212 \times 10^{-1}$
27	1.0280086×10^0	$2.6542635452061003 \times 10^{-1}$	$7.62582226 \times 10^{-1}$
28	9.4162940×10^{-1}	$8.503519690601384 \times 10^{-1}$	$9.12774407 \times 10^{-2}$
29	1.1065198×10^0	$1.2321124623871897 \times 10^0$	$1.25592644 \times 10^{-1}$
30	7.5292090×10^{-1}	$3.7414648963928676 \times 10^{-1}$	$3.78774410 \times 10^{-1}$
31	1.3110139×10^0	$1.0766291714289444 \times 10^0$	$2.34384766 \times 10^{-1}$
32	8.7783100×10^{-2}	$8.291255674004515 \times 10^{-1}$	$7.41342469 \times 10^{-1}$
33	3.2801480×10^{-1}	$1.2541546500504441 \times 10^0$	$9.26139859 \times 10^{-1}$
34	9.8927810×10^{-1}	$2.9790694147232066 \times 10^{-1}$	$6.91371137 \times 10^{-1}$
35	1.0210990×10^0	$9.253821285571046 \times 10^{-1}$	$9.57168428 \times 10^{-2}$
36	9.5646656×10^{-1}	$1.1325322626697856 \times 10^0$	$1.76065707 \times 10^{-1}$
37	1.0813814×10^0	$6.822410727153098 \times 10^{-1}$	$3.99140367 \times 10^{-1}$
38	8.1736827×10^{-1}	$1.3326056469620293 \times 10^0$	$5.15237378 \times 10^{-1}$
39	1.2652004×10^0	$2.9091569028512065 \times 10^{-3}$	1.26229122×10^0
40	2.5860548×10^{-1}	$1.1611238029748606 \times 10^{-2}$	$2.46994242 \times 10^{-1}$

5.4 Wnioski

Rozważany model logistyczny dla przyjętych parametrów jest **bardzo niestabilny numerycznie**. Zarówno celowe obcięcie wyniku w kroku $n = 10$, jak i same błędy zaokrągleń (F32 vs F64) powodują dużą rozbieżność wyników.

6 Zadanie 6: Analiza rekurencji $x_{n+1} = x_n^2 + c$

6.1 Opis problemu

Celem zadania była obserwacja zachowania ciągu generowanego przez równanie rekurencyjne $x_{n+1} = x_n^2 + c$ dla siedmiu różnych kombinacji stałej c oraz warunku początkowego x_0 .

6.2 Rozwiązanie

Równanie iterowałam 40 razy dla każdego z siedmiu przypadków. Przeprowadziłam też iterację graficzną za pomocą wykresów, które ilustrują, jak zachowuje się ciąg.

6.3 Wyniki i interpretacja

Wyniki podzieliłam na dwie grupy: przypadki stabilne i cykliczne oraz pozostałe przypadki w celu bardziej zwartego zapisu tabel.

6.3.1 Case 1, 2, 4, 5: Przypadki stabilne i cykliczne

Te przypadki od razu (lub bardzo szybko) osiągają punkt stały lub stabilny cykl 2-elementowy.

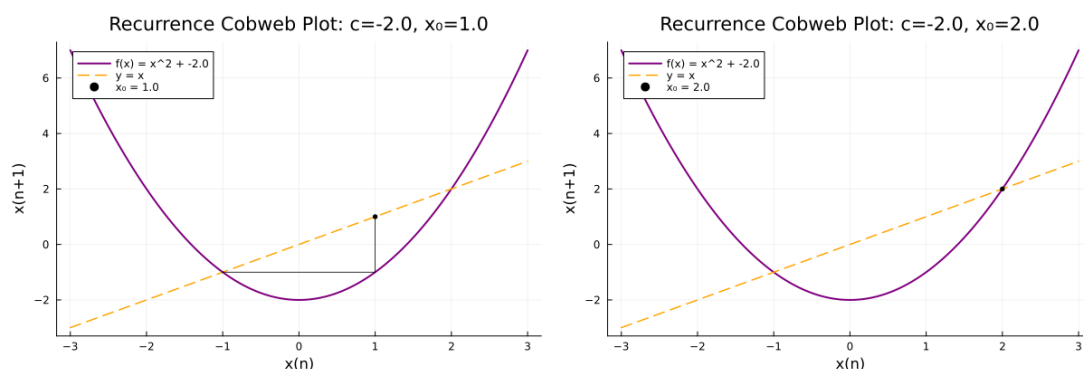


Figure 2: Wykresy (Cobweb plots) dla przypadków stabilnych

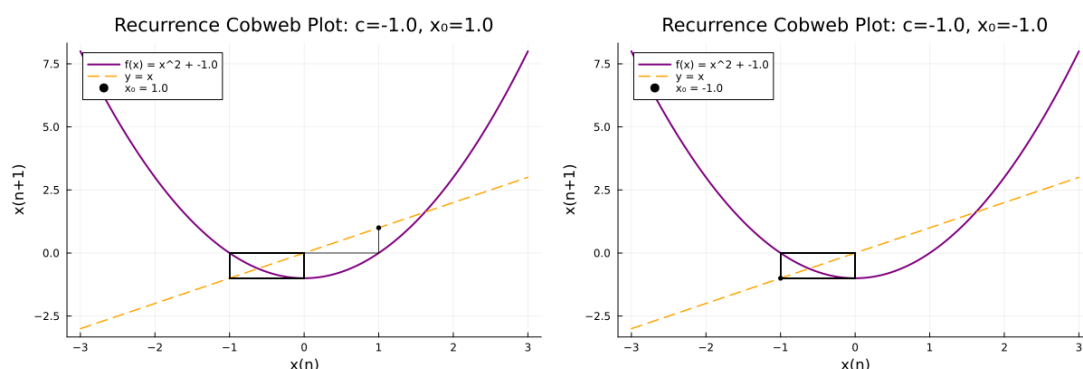


Figure 3: Wykresy (Cobweb plots) dla przypadków cyklicznych

Table 9: Wyniki dla przypadków stabilnych i cyklicznych (wybrane iteracje)

n	Case 1 ($c = -2, x_0 = 1$)	Case 2 ($c = -2, x_0 = 2$)	Case 4 ($c = -1, x_0 = 1$)	Case 5 ($c = -1, x_0 = -1$)
0	1.0	2.0	1.0	-1.0
1	-1.0	2.0	0.0	0.0
2	-1.0	2.0	-1.0	-1.0
3	-1.0	2.0	0.0	0.0
4	-1.0	2.0	-1.0	-1.0
5	-1.0	2.0	0.0	0.0
6	-1.0	2.0	-1.0	-1.0
...	...	...	...	...
39	-1.0	2.0	0.0	0.0
40	-1.0	2.0	-1.0	-1.0

- **Case 1:** ciąg, startując z $x_0 = 1$, natychmiast wpada w punkt $x' = -1$ i go nie opuszcza.
- **Case 2:** ciąg startuje z $x_0 = 2$ i cały czas w nim pozostaje.
- **Case 4 i 5:** wartości wpadają w cykl, na zmianę przyjmując wartości -1 i 0 .

6.3.2 Case 3, 6, 7: Przypadki dynamiczne

Te przypadki ilustrują wartości zmieniające się bardziej dynamicznie, czasem chaotycznie.

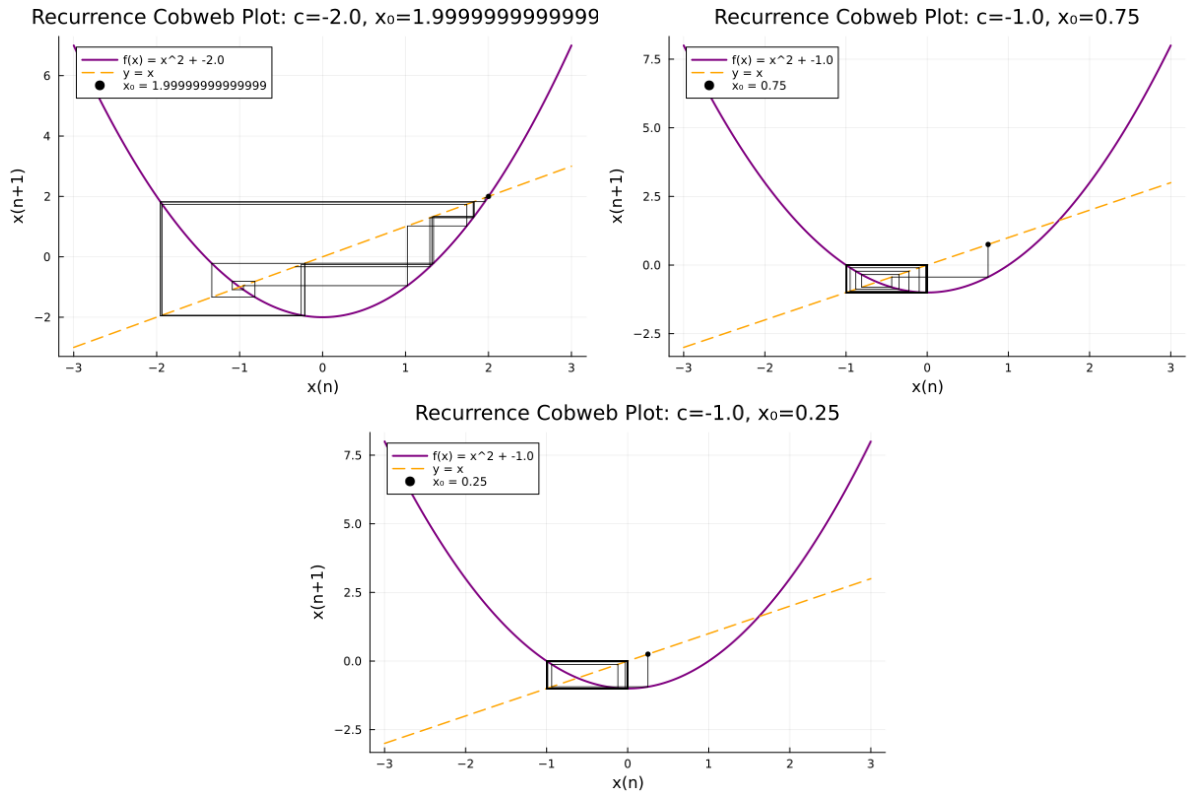


Figure 4: Wykresy (Cobweb plots) dla przypadków dynamicznych

Table 10: Wyniki dla przypadków dynamicznych

n	Case 3 ($c = -2, x_0 = 1.99\dots$)	Case 6 ($c = -1, x_0 = 0.75$)	Case 7 ($c = -1, x_0 = 0.25$)
0	1.99999999999999	0.75	0.25
1	1.99999999999996	-0.4375	-0.9375
2	1.9999999999998401	-0.80859375	-0.12109375
3	1.9999999999993605	-0.3461761474609375	-0.9853363037109375
4	1.999999999997442	-0.8801620749291033	-0.029112368589267135
5	1.999999999897682	-0.2253147218564956	-0.9991524699951226
6	1.999999999590727	-0.9492332761147301	-0.0016943417026455965
7	1.99999999836291	-0.0989561875164966	-0.9999971292061947
8	1.999999993451638	-0.9902076729521999	$-5.741579369278327 \times 10^{-6}$
9	1.999999973806553	-0.01948876442658909	-0.999999999670343
10	1.999999989522621	-0.999620188061125	$-6.593148249578462 \times 10^{-11}$
11	1.9999999580904841	-0.0007594796206411569	-1.0
12	1.9999998323619383	-0.9999994231907058	0.0
13	1.9999993294477814	$-1.1536182557003727 \times 10^{-6}$	-1.0
14	1.9999973177915749	-0.999999999986692	0.0
15	1.9999892711734937	$-2.6616486792363503 \times 10^{-12}$	-1.0
16	1.9999570848090826	-1.0	0.0
17	1.999828341078044	0.0	-1.0
18	1.9993133937789613	-1.0	0.0
19	1.9972540465439481	0.0	-1.0
20	1.9890237264361752	-1.0	0.0
21	1.9562153843260486	0.0	-1.0
22	1.82677862987391	-1.0	0.0
23	1.3371201625639997	0.0	-1.0
24	-0.21210967086482313	-1.0	0.0
25	-1.9550094875256163	0.0	-1.0
26	1.822062096315173	-1.0	0.0
27	1.319910282828443	0.0	-1.0
28	-0.2578368452837396	-1.0	0.0
29	-1.9335201612141288	0.0	-1.0
30	1.7385002138215109	-1.0	0.0
31	1.0223829934574389	0.0	-1.0
32	-0.9547330146890065	-1.0	0.0
33	-1.0884848706628412	0.0	-1.0
34	-0.8152006863380978	-1.0	0.0
35	-1.3354478409938944	0.0	-1.0
36	-0.21657906398474625	-1.0	0.0
37	-1.953093509043491	0.0	-1.0
38	1.8145742550678174	-1.0	0.0
39	1.2926797271549244	0.0	-1.0
40	-0.3289791230026702	-1.0	0.0

- **Case 3:** Początkowy, minimalny błąd jest wzmacniany w każdej iteracji. W szczególności po ok. 20-21 kroku następuje gwałtowne zmiana wartości i wejście w chaotyczne zmiany wyników.
- **Case 6 i 7:** Oba przypadki pokazują początkowe błędzenie, które ostatecznie kończy się wpadnięciem do stabilnego cyklu 2-elementowego (z elementami 0 i -1).

6.4 Wnioski

1. Zachowanie ciągu zależy od parametrów wejściowych (tu: c i x_0). W zależności od tych wartości, ciąg może:
 - Zbiegać do jednej, stałej wartości (jak w Case 1).
 - Stabilizować się w stałym cyklu (jak w Case 4, 5, 6, 7).
 - Zachowywać się chaotycznie (jak w Case 3).
2. Dla $c = -1$, we wszystkich rozpatrywanych przypadkach (Case 4, 5, 6, 7) pomimo różnych punktów startowych ciąg ostatecznie wpada w ten sam cykl.
3. Dla $c = -2$, gdy startujemy w punkcie 1 lub 2 (Case 1 i 2), otrzymujemy ciągi stałe. W Case 3 już minimalny błąd (różnica 10^{-14}) wystarczył, aby ciąg wpadł w chaos.